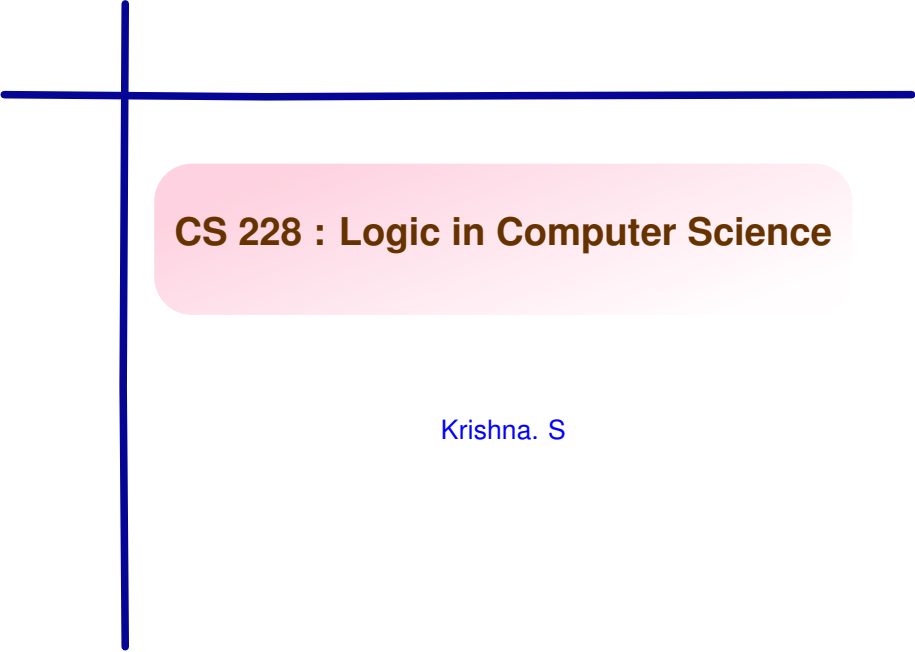


A decorative blue crosshair consisting of a vertical line and a horizontal line intersecting in the upper-left quadrant of the slide.

CS 228 : Logic in Computer Science

Krishna. S

Recap : Translation Lemma

Translation Lemma

If t is a term and F is a formula such that no variable in t occurs bound in F , then $\mathcal{A} \models_{\alpha} F[t/x]$ iff $\mathcal{A} \models_{\alpha[x \mapsto \alpha(t)]} F$.

$F[t/x]$ denotes substituting t in place of x in F , where x is free in F

- ▶ What if t contains a variable bound in F ?
- ▶ Results in *Variable Capture*
- ▶ We saw examples of variable capture in the last class

Recap : Renaming

$\int_0^\infty f(s)ds$ has the same value as $\int_0^\infty f(t)dt$

Renaming Lemma

Let $F = Qx[G]$ be a formula with $Q \in \{\exists, \forall\}$. Let y be a variable which does not appear in G . Then $\mathcal{A} \models_\alpha F$ iff $\mathcal{A} \models_\alpha Qy(G[y/x])$.

Assume $Q = \forall$.

$\mathcal{A} \models_\alpha \forall y G[y/x]$ iff $\mathcal{A} \models_{\alpha[y \mapsto a]} G[y/x]$ for all $a \in U^{\mathcal{A}}$
iff $\mathcal{A} \models_{\alpha[y \mapsto a, x \mapsto \alpha[y \mapsto a](y)]} G$ for all $a \in U^{\mathcal{A}}$
(Translation Lemma)
iff $\mathcal{A} \models_{\alpha[y \mapsto a, x \mapsto a]} G$ for all $a \in U^{\mathcal{A}}$
iff $\mathcal{A} \models_{\alpha[x \mapsto a]} G$ for all $a \in U^{\mathcal{A}}$
iff $\mathcal{A} \models_\alpha \forall x G$

Recap : Rectified Formulae

A FOL formula is *rectified* if no variable occurs both free and bound, and if all quantifiers in the formula refer to different variables.

$$\forall x \exists y P(x, f(y)) \wedge \forall y (Q(x, y) \vee R(x))$$

is not rectified.

Recap : Rectified Formulae

A FOL formula is *rectified* if no variable occurs both free and bound, and if all quantifiers in the formula refer to different variables.

$$\forall x \exists y P(x, f(y)) \wedge \forall y (Q(x, y) \vee R(x))$$

is not rectified. By renaming we obtain an equivalent rectified formula

$$\forall u \exists v P(u, f(v)) \wedge \forall y (Q(x, y) \vee R(x))$$

By Renaming Lemma, we can always obtain an equivalent rectified formula by renaming bound variables.

Recap : Prenex Normal Form and Rectified Prenex Normal form (RPF)

A formula is in prenex normal form if it can be written as

$$Q_1 x_1 Q_2 x_2 \dots Q_n x_n F$$

where $Q_i \in \{\forall, \exists\}$, $n \geq 0$ and F has no quantifiers. F is called the matrix of the formula.

- ▶ Given a rectified formula, we can use the equivalences from the last lecture to convert F into rectified, prenex normal form, by “pushing all quantifiers up front”.
- ▶ Otherwise, rectify the formula first, and then convert to prenex normal form.

Skolemisation

A sentence in RPF is in *Skolem form* if it has no occurrences of the existential quantifier.

We can transform any sentence in RPF to an equisatisfiable formula in Skolem form by using extra function symbols.

- ▶ $\exists x \forall y \exists z \exists w G(x, y, z, w)$ is equisatisfiable with $\forall y G(c, y, f(y), g(y))$ where c is a constant.

Skolemisation

Skolem Lemma

Let $F = \forall y_1 \forall y_2 \dots \forall y_n \exists z G$ be in RPF. Given a function symbol f of arity n which does not appear in F , write

$$F' = \forall y_1 \forall y_2 \dots \forall y_n G[f(y_1, \dots, y_n)/z]$$

Then F and F' are equisatisfiable.

One direction : Assume F is satisfiable. Let $\mathcal{A} \models_{\alpha} F$.

- ▶ Key Idea : Extend structure $\mathcal{A}$ with an interpretation for a function f such that $\mathcal{A}' \models_{\alpha'} F'$.
- ▶ Given any $a_1, \dots, a_n \in U^{\mathcal{A}}$, choose $a \in U^{\mathcal{A}}$ such that $\mathcal{A} \models_{\alpha[y_1 \mapsto a_1, \dots, y_n \mapsto a_n, z \mapsto a]} G$, and define $f^{\mathcal{A}'}(a_1, \dots, a_n) = a$.
- ▶ f does not appear in G , $\mathcal{A}' \models_{\alpha[y_1 \mapsto a_1, \dots, y_n \mapsto a_n, z \mapsto f^{\mathcal{A}'}(a_1, \dots, a_n)]} G$,
- ▶ By Translation Lemma, $\mathcal{A}' \models_{\alpha[y_1 \mapsto a_1, \dots, y_n \mapsto a_n]} G[f(y_1, \dots, y_n)/z]$
- ▶ Since this holds for any $a_1, \dots, a_n \in U^{\mathcal{A}}$,
 $\mathcal{A}' \models \forall y_1 \forall y_2 \dots \forall y_n G[f(y_1, \dots, y_n)/z]$

Skolemisation : Example

$$\forall x \exists y \forall z \exists w (\neg P(a, w) \vee Q(f(x), y))$$

- ▶ By Skolem Lemma, eliminate $\exists y$ and introduce a new function g , obtaining $\forall x \forall z \exists w (\neg P(a, w) \vee Q(f(x), g(x)))$
- ▶ By Skolem Lemma, eliminate $\exists w$ introducing a new function h obtaining $\forall x \forall z (\neg P(a, h(x, z)) \vee Q(f(x), g(x)))$

Conversion to Skolem Form :

Summary

Convert an arbitrary FOL formula to an equisatisfiable formula in Skolem form as follows:

1. Rectify F systematically renaming bound variables, obtaining an equivalent formula F_1
2. Use the equivalences in the beginning and move all quantifiers outside, yielding an equivalent formula F_2 in prenex normal form
3. Repeatedly eliminate the outermost existential quantifier in F_2 until an equisatisfiable formula F_3 is obtained in Skolem form.

Semi Decidability of Satisfiability

- ▶ Given a FOL formula in Skolem normal form, if F is unsatisfiable, there is a technique of *Ground Resolution* which gives $\perp$ and terminates.
- ▶ However, if F is satisfiable, then this process may go on forever.
- ▶ Validity is *semi decidable* : a valid formula F has a finite witness of its validity, namely, a finite resolution refutation for $\neg F$.
- ▶ If F is not valid, but satisfiable, then there may not be a finite witness.
- ▶ This is for general FOL : however, we can focus on FOL over some special signatures where satisfiability is decidable.

Satisfaction, Validity

- ▶ Given a FO formula $\varphi(x_1, \dots, x_n)$ over a signature τ , is it satisfiable/valid?
 - ▶ Satisfiable, if there exists a τ -structure $\mathcal{A}$ and an assignment α for $x_1, \dots, x_n$ in $u(\mathcal{A})$ such that $\mathcal{A} \models_{\alpha} \varphi(x_1, \dots, x_n)$

Satisfaction, Validity

- ▶ Given a FO formula $\varphi(x_1, \dots, x_n)$ over a signature τ , is it satisfiable/valid?
 - ▶ Satisfiable, if there exists a τ -structure $\mathcal{A}$ and an assignment α for $x_1, \dots, x_n$ in $u(\mathcal{A})$ such that $\mathcal{A} \models_{\alpha} \varphi(x_1, \dots, x_n)$
 - ▶ Valid, if for any τ -structure $\mathcal{A}$ and any assignment α for $x_1, \dots, x_n$ in $u(\mathcal{A})$, $\mathcal{A} \models_{\alpha} \varphi(x_1, \dots, x_n)$
- ▶ Assume we fix the type of the structure $\mathcal{A}$, say words (why words?)

FOL over Words

- ▶ Given an FO sentence φ over words, is it satisfiable/valid?
- ▶ Satisfiable (Valid) iff some word (all words) satisfies φ

FOL over Words

- ▶ Given an FO sentence φ over words, is it satisfiable/valid?
- ▶ Satisfiable (Valid) iff some word (all words) satisfies φ
- ▶ There could be infinitely many words w satisfying φ
- ▶ $L(\varphi) = \{ \text{words } w \mid w \models \varphi \}$ is called the language of φ

FOL over Words

- ▶ Given an FO sentence φ over words, is it satisfiable/valid?
- ▶ Satisfiable (Valid) iff some word (all words) satisfies φ
- ▶ There could be infinitely many words w satisfying φ
- ▶ $L(\varphi) = \{ \text{words } w \mid w \models \varphi \}$ is called the language of φ
- ▶ Given φ , write an algorithm to check $L(\varphi) = \emptyset$

First-Order Logic over Words

Expressiveness and Satisfiability

- ▶ Signature for words : $<$, S and Q_a for finitely many symbols a
- ▶ Given a FO formula over words, the signature is fixed

Expressiveness and Satisfiability

- ▶ Signature for words : $<$, S and Q_a for finitely many symbols a
- ▶ Given a FO formula over words, the signature is fixed
- ▶ **Expressiveness**
 - ▶ Given a set of words or a language L , **can you** write a FO formula φ such that $L(\varphi) = L$

Expressiveness and Satisfiability

- ▶ Signature for words : $<$, S and Q_a for finitely many symbols a
- ▶ Given a FO formula over words, the signature is fixed
- ▶ **Expressiveness**
 - ▶ Given a set of words or a language L , **can you** write a FO formula φ such that $L(\varphi) = L$
 - ▶ If you can, FO is expressive enough to capture your language/specification/property

Expressiveness and Satisfiability

- ▶ Signature for words : $<$, S and Q_a for finitely many symbols a
- ▶ Given a FO formula over words, the signature is fixed
- ▶ **Expressiveness**
 - ▶ Given a set of words or a language L , **can you** write a FO formula φ such that $L(\varphi) = L$
 - ▶ If you can, FO is expressive enough to capture your language/specification/property
 - ▶ If you cannot, show that FO cannot capture your property.
- ▶ **Satisfiability**

Expressiveness and Satisfiability

- ▶ Signature for words : $<$, S and Q_a for finitely many symbols a
- ▶ Given a FO formula over words, the signature is fixed
- ▶ **Expressiveness**
 - ▶ Given a set of words or a language L , **can you** write a FO formula φ such that $L(\varphi) = L$
 - ▶ If you can, FO is expressive enough to capture your language/specification/property
 - ▶ If you cannot, show that FO cannot capture your property.
- ▶ **Satisfiability**
 - ▶ Given a FO formula φ over words, is $L(\varphi)$ non-empty?

A Primer for Words

Alphabet

- ▶ An alphabet Σ is a finite set
 - ▶ $\Sigma = \{a, b, \dots, z\}$
 - ▶ $\Sigma = \{+, \alpha, 100, B\}$
- ▶ Elements of Σ called letters or symbols
- ▶ A word or string over Σ is a finite sequence of symbols from Σ
- ▶ If $\Sigma = \{a, b\}$, then *abababa* is a word of length 7
- ▶ The length of a word w is denoted $|w|$
- ▶ There is a unique word of length 0 denoted ϵ , called the empty word
- ▶ $|\epsilon| = 0$

Notations for Words

- ▶ $aaaaa$ denoted a^5
- ▶ $a^0 = \epsilon$
- ▶ $a^{n+1} = a^n.a = a.a^n$
- ▶ The set of all words over Σ is denoted Σ^*
 - ▶ $\{a, b\}^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$
 - ▶ $\{a\}^* = \{\epsilon, a, aa, aaa, \dots\} = \{a^n \mid n \geq 0\}$
- ▶ By convention, $\{\}^* = \{\epsilon\}$

Notations for Words

- ▶ Σ is a finite set
- ▶ Σ^* is the infinite set of all finite words over alphabet Σ
- ▶ Each $w \in \Sigma^*$ is a finite word
 - ▶ $\{a, b\} = \{b, a\}$ but $ab \neq ba$
 - ▶ $\{a, a, b\} = \{a, b\}$ but $aab \neq ab$
 - ▶ $\emptyset$ is the set consisting of no words
 - ▶ $\{\epsilon\}$ is a set having the single word ϵ
 - ▶ ϵ is a word

Operations on Words

- ▶ Concatenation of words : $x.y = xy$
 - ▶ Concatenation is associative : $x.(yz) = (xy).z$
 - ▶ Concatenation not commutative in general $x.y \neq y.x$
 - ▶ ϵ is the identity for concatenation $\epsilon.x = x.\epsilon = x$
 - ▶ $|x.y| = |x| + |y|$
- ▶ x^n : catenating word x n times
 - ▶ $(aab)^5 = \text{aab aab aab aab aab}$
 - ▶ $(aab)^0 = \epsilon$
 - ▶ $(aab)^* = \{\epsilon, aab, aabaab, aabaabaab, \dots\}$
 - ▶ $x^{n+1} = x^n x$

More Operations on Words

- ▶ For $a \in \Sigma$ and $x \in \Sigma^*$,

$|x|_a =$ *number of times the symbol a occurs in the word x*

- ▶ $|aabbaa|_a = 4, |aabbaa|_b = 2$
- ▶ $|\epsilon|_a = 0$
- ▶ Prefix of a word $w \in \Sigma^*$ is an initial subword of w

$$\text{Pref}(w) = \{x \in \Sigma^* \mid \exists y \in \Sigma^*, w = x.y\}$$

- ▶ $\text{Pref}(aaba) = \{\epsilon, a, aa, aab, aaba\}$
- ▶ Proper prefixes = $\{a, aa, aab\}$
- ▶ $\epsilon, aaba$ improper prefixes

Operation on Sets

Given a finite alphabet Σ , denote by $A, B, C, \dots$ subsets of Σ^*

- ▶ Subsets of Σ^* are called languages
- ▶ $A \cup B = \{x \in \Sigma^* \mid x \in A \text{ or } x \in B\}$
 - ▶ $A = a^*, B = \{b, bb\}, A \cup B = a^* \cup \{b, bb\}$
- ▶ $A \cap B = \{x \in \Sigma^* \mid x \in A \text{ and } x \in B\}$
 - ▶ $A = (ab)^*, B = \{abab, \epsilon, bb\}, A \cap B = \{\epsilon, abab\}$
- ▶ $\bar{A} = \{x \in \Sigma^* \mid x \notin A\}$
 - ▶ For $\Sigma = \{a\}$ and $A = (aa)^*, \bar{A} = \{a, a^3, a^5, \dots\}$
- ▶ $AB = \{xy \mid x \in A, y \in B\}$
 - ▶ $A = \{a, ba\}, B = \{\epsilon, aa, bb\}$
 - ▶ $AB = \{a, a^3, abb, ba, ba^3, babb\}$
 - ▶ $BA = \{a, ba, a^3, aaba, bba, bbba\}$

Operation on Sets

For a set $A \subseteq \Sigma^*$,

- ▶ $A^0 = \{\epsilon\}$
- ▶ $A^{n+1} = A.A^n$
 - ▶ $\{a, ab\}^2 = \{a, ab\}.\{a, ab\} = \{aa, aab, aba, abab\}$
 - ▶ $\{a, b\}^n = \{x \in \{a, b\}^* \mid |x| = n\}$
- ▶ $A^* = A^0 \cup A \cup A^2 \cup \dots = \bigcup_{i \geq 0} A^i$
- ▶ $A^+ = AA^* = A \cup A^2 \cup \dots = \bigcup_{i \geq 1} A^i$
- ▶ Union : Associative, commutative
- ▶ Concatenation : Associative, Non commutative
- ▶ $A \cup \emptyset = \emptyset \cup A = A$
- ▶ $\{\epsilon\}A = A\{\epsilon\} = A$
- ▶ $\emptyset A = A\emptyset = \emptyset$

Operation on Sets

- ▶ Union, Intersection distribute over union
 - ▶ $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
 - ▶ $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
- ▶ Concatenation distributes over union
 - ▶ $A(\cup_{i \in I} B_i) = \cup_{i \in I} AB_i$
 - ▶ $(\cup_{i \in I} B_i)A = \cup_{i \in I} B_iA$
- ▶ Concatenation does not distribute over intersection
 - ▶ $A = \{a, ab\}, B = \{b\}, C = \{\epsilon\}$
 - ▶ $A(B \cap C) \neq AB \cap AC$

FO for Languages

Formalize in FO

Write FO formulae φ_i such that $L(\varphi_i) = L_i$ for $i = 1, \dots, 5$.

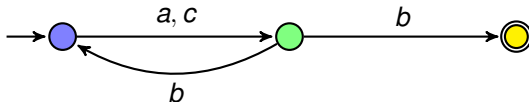
- ▶ L_1 = Words that have exactly one occurrence of the letter c
- ▶ L_2 = Words that begin with a and end with b
- ▶ L_3 = Words that have no two consecutive a 's
- ▶ L_4 = Words in which any a is followed immediately by a b
- ▶ L_5 = Words in which whenever an a occurs, it is followed eventually by a b , and no c occurs in between the a and the b
 $aabbabab, aabbcbbccaab \in L_5, aacaab \notin L_5$.

Satisfiability of FO over Words

- ▶ Recall : Given an FO sentence φ over words, is $L(\varphi) = \emptyset$?
- ▶ Algorithm?

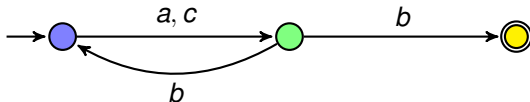
Core Idea

- ▶ Given FO formula φ over an alphabet Σ , construct an **edge labeled graph** G_φ : a graph whose edges are **labeled** by Σ .



Core Idea

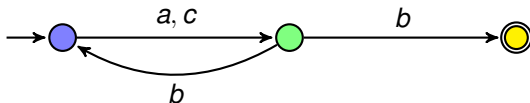
- ▶ Given FO formula φ over an alphabet Σ , construct an **edge labeled graph** G_φ : a graph whose edges are **labeled** by Σ .



- ▶ Each path in the graph gives rise to a word over Σ , obtained by reading off the labels on the edges

Core Idea

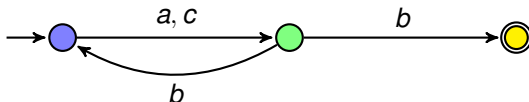
- ▶ Given FO formula φ over an alphabet Σ , construct an **edge labeled graph** G_φ : a graph whose edges are **labeled** by Σ .



- ▶ Each path in the graph gives rise to a word over Σ , obtained by reading off the labels on the edges
- ▶ G_φ has some **special** kinds of vertices
 - ▶ There is a unique vertex called the **start** vertex (blue vertex)
 - ▶ There are some vertices called **good** vertices (yellow vertex)

Core Idea

- ▶ Given FO formula φ over an alphabet Σ , construct an **edge labeled graph** G_φ : a graph whose edges are **labeled** by Σ .



- ▶ Each path in the graph gives rise to a word over Σ , obtained by reading off the labels on the edges
- ▶ G_φ has some **special** kinds of vertices
 - ▶ There is a unique vertex called the **start** vertex (blue vertex)
 - ▶ There are some vertices called **good** vertices (yellow vertex)
- ▶ Read off words on paths from the start vertex to any final vertex and call this set of words $L(G_\varphi)$
- ▶ Ensure that G_φ is constructed such that $L(\varphi) = L(G_\varphi)$.

Core Idea

- ▶ Why does this help?

Core Idea

- ▶ Why does this help?
- ▶ We know how to check the existence of a path between 2 vertices in a graph **easily**

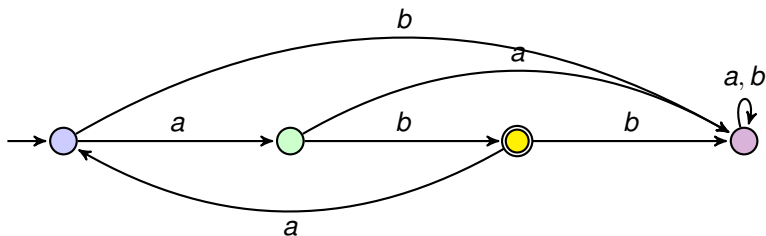
Core Idea

- ▶ Why does this help?
- ▶ We know how to check the existence of a path between 2 vertices in a graph **easily**
- ▶ If **somehow** we manage to construct G_φ **correctly**, then checking satisfiability of φ is same as checking the **reachability** of some good vertex from the start vertex of G_φ .

Core Idea

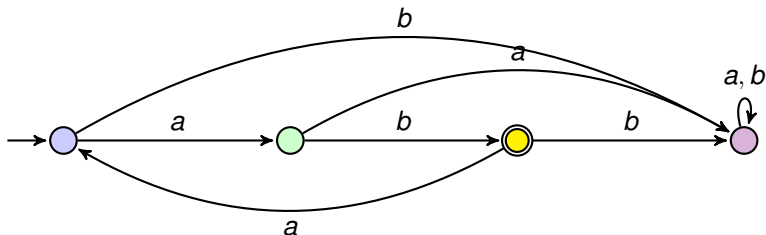
- ▶ Why does this help?
- ▶ We know how to check the existence of a path between 2 vertices in a graph **easily**
- ▶ If **somehow** we manage to construct G_φ **correctly**, then checking satisfiability of φ is same as checking the **reachability** of some good vertex from the start vertex of G_φ .
- ▶ How to construct G_φ ?

A First Machine A



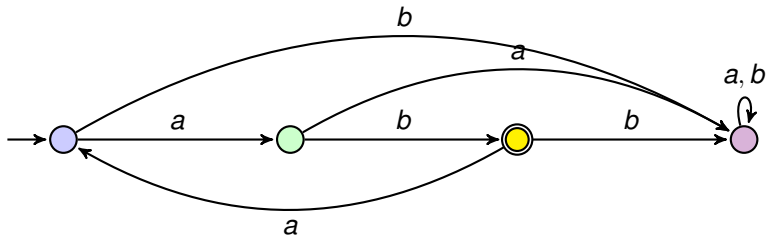
- ▶ Colored circles called **states**
- ▶ Arrows between circles called **transitions**
- ▶ Blue state called an **initial state**
- ▶ Doubly circled state called a **final state**

A First Machine A



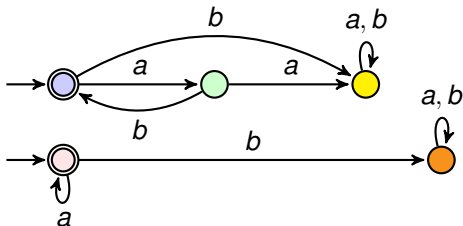
- ▶ A path from one state to another gives a word over $\Sigma = \{a, b, c\}$
- ▶ The machine **accepts** words along paths from an initial state to a final state
- ▶ The set of words accepted by the machine is called the **language** accepted by the machine

A First Machine A



- ▶ What is the language L accepted by this machine, $L(A)$?
- ▶ Write an FO formula φ such that $L(\varphi) = L(A)$

A Second and a Third Machine B, C



- ▶ What are $L(B)$, $L(C)$?
- ▶ Give an FO formula φ such that $L(\varphi) = L(B) \cup L(C)$

Finite State Machines

A deterministic finite state automaton (DFA) $A = (Q, \Sigma, \delta, q_0, F)$

- ▶ Q is a finite set of states
- ▶ Σ is a finite alphabet
- ▶ $\delta : Q \times \Sigma \rightarrow Q$ is the transition function
- ▶ $q_0 \in Q$ is the initial state
- ▶ $F \subseteq Q$ is the set of final states
- ▶ $L(A)$ =all words leading from q_0 to some $f \in F$

Languages, Machines and Logic

A language $L \subseteq \Sigma^*$ is called **regular** iff there exists some DFA A such that $L = L(A)$.

A language $L \subseteq \Sigma^*$ is called **FO-definable** iff there exists an FO formula φ such that $L = L(\varphi)$.

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Begins with a , ends with b , and has a pair of consecutive a 's
- ▶ Contains a b and ends with aa
- ▶ Contains abb
- ▶ There are two occurrences of b between which only a 's occur
- ▶ Right before the last position is an a
- ▶ Even length words

Is it Regular? Is it FO-definable?

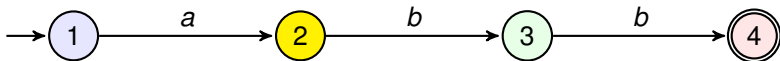
$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- Contains *abb*

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- Contains *abb*

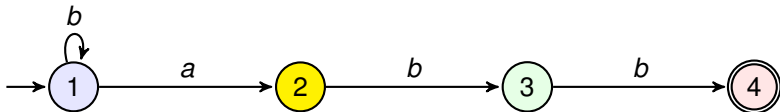


$$\exists x \exists y \exists z (Q_a(x) \wedge Q_b(y) \wedge Q_b(z) \wedge S(x, y) \wedge S(y, z))$$

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

► Contains *abb*

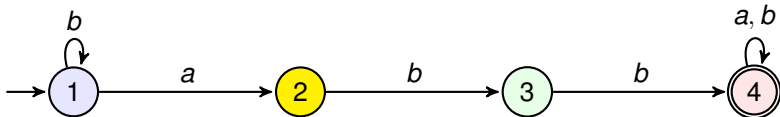


$$\exists x \exists y \exists z (Q_a(x) \wedge Q_b(y) \wedge Q_b(z) \wedge S(x, y) \wedge S(y, z))$$

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- Contains *abb*

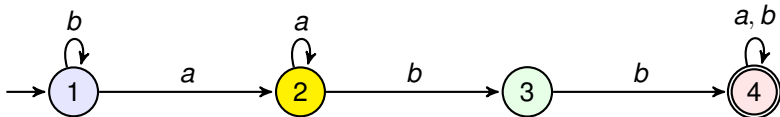


$$\exists x \exists y \exists z (Q_a(x) \wedge Q_b(y) \wedge Q_b(z) \wedge S(x, y) \wedge S(y, z))$$

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

► Contains *abb*

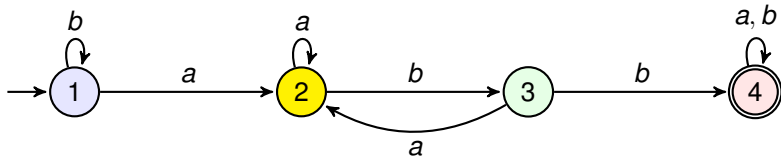


$$\exists x \exists y \exists z (Q_a(x) \wedge Q_b(y) \wedge Q_b(z) \wedge S(x, y) \wedge S(y, z))$$

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

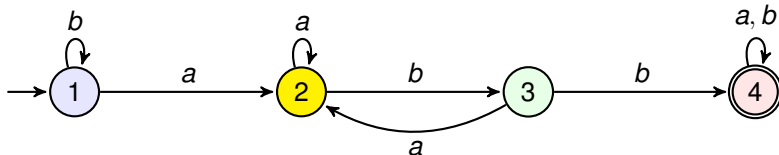
- Contains *abb*



Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

► Contains *abb*



$$\exists x \exists y \exists z (Q_a(x) \wedge Q_b(y) \wedge Q_b(z) \wedge S(x, y) \wedge S(y, z))$$

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$

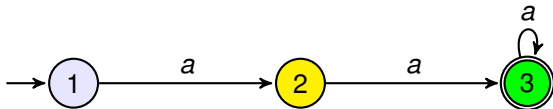
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$

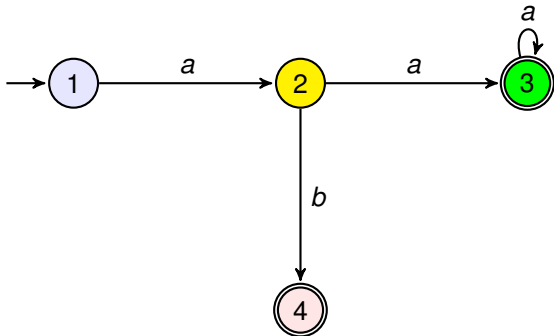
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$

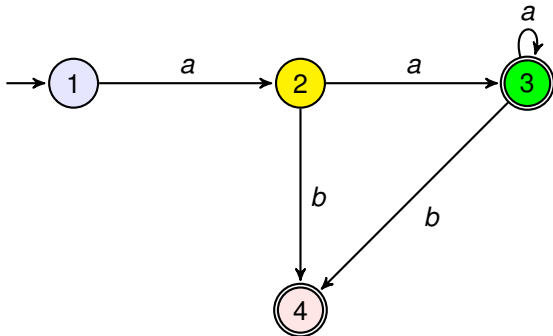
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$

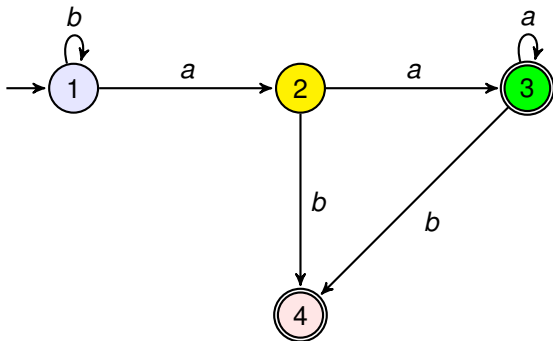
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$

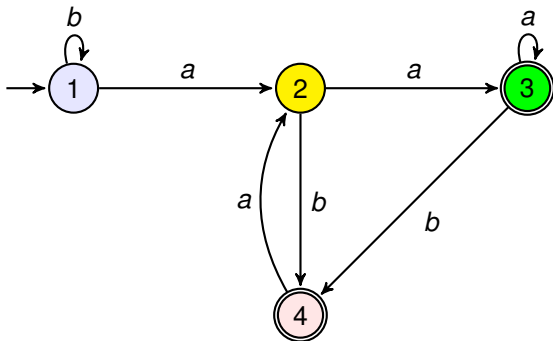
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$

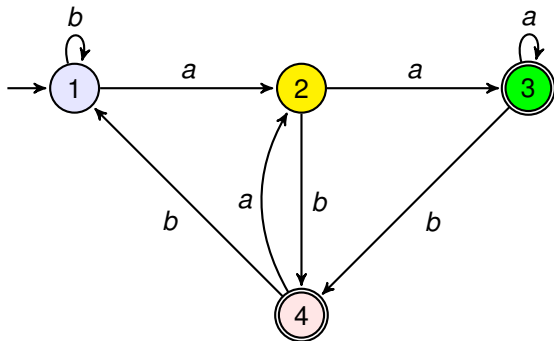
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



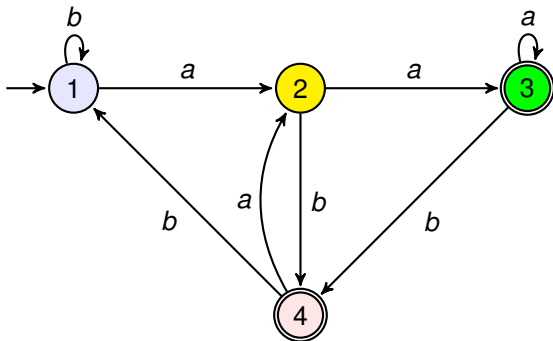
Is it Regular? Is it FO-definable?

$\Sigma = \{a, b\}$. Consider the following languages $L \subseteq \Sigma^*$:

- ▶ Right before the last position is an a :

Examples : $ab, babbaa, bbab$

Non examples : ba, bb, aba



$$\exists x[Q_a(x) \wedge \exists y(S(x, y) \wedge \forall z(z \leq y))]$$